C# and .NET Coding Standard

Contents

C# and .NET Coding Standard **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 3
- References . . . 3
- Related . . . 3

C# and .NET Coding Standard

Scope

Language-level and project-file conventions for the .NET 10 backend. Architecture-level rules are in `../clean-architecture-cqrs/STANDARD.md`.

Official requirements

Microsoft's published guidance is the baseline; the project does not invent a competing style:

- .NET runtime and C# language conventions (naming, layout, `var` usage) — Microsoft C# coding conventions.
- Framework design guidelines for public API surface naming.
- Nullable reference types are a language feature the project enables; warnings from it are real defects.

Compliance is enforced, not requested. `IDE1006` (naming rule violation) is escalated to an error, and `TreatWarningsAsErrors` is on, so a deviation from Microsoft's naming conventions fails the build. Decision: `brain/decisions/ADR-0009-code-quality-gates.md`.

Project policy

Project files

- `TargetFramework` is `net10.0` for every project. One version, no per-project drift.
- Enable in every `.csproj`: `<Nullable>enable</Nullable>`, `<ImplicitUsings>enable</ImplicitUsings>`, `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>`.
- Central package management via `Directory.Packages.props`. A version number appears once in the solution. Never copy versions from `reference/nArchitecture` — that tree is net6 and stale.
- The dependency set is fixed by `brain/decisions/ADR-0021-backend-dependency-set.md`: MediatR, FluentValidation, `Microsoft.AspNetCore.RateLimiting`, `Microsoft.AspNetCore.OpenApi`, Serilog + OpenTelemetry, `SonarAnalyzer.CSharp`, `BCrypt.Net-Next`, `Npgsql.EntityFrameworkCore.PostgreSQL`, xUnit + NSubstitute + Testcontainers. **Adding a package is a decision, not a convenience** — it needs an ADR.
- **No mapping library.** Project to DTOs with `Select` inside the query.

- A single `Directory.Build.props` carries shared properties; individual `.csproj` files stay short.

Naming

Element	Convention
namespace, type, method, property, const	`PascalCase`
parameter, local	`camelCase`
private field	`_camelCase`
interface	`IPascalCase`
async method	suffix `Async`
CQRS types	`<Verb><Noun>Command` / `<Verb><Noun>Query`, handler `<Name>Handler`

File name matches the single public type it contains. One public type per file.

Language use

- `async`/`await` end to end. No `.Result`, no `.Wait()`, no `async void` outside event handlers.
- Every async method that can be cancelled takes a `CancellationToken` and passes it down.
- Prefer records for DTOs and immutable value objects; classes for entities with identity.
- No `#region`. If a file needs regions it needs splitting.
- Exceptions are for exceptional flow. Expected failures (validation, not-found, forbidden) return typed results or throw the project's own domain exception types that middleware maps to HTTP status codes.
- No business logic in constructors and none in controllers.
- `DateTime.UtcNow` only — never `DateTime.Now`. Inject a clock abstraction where the value is asserted in tests.

Size and complexity limits

Enforced by analyzers, configured once in the root `.editorconfig`. A limit that lives only in a document is a suggestion, and suggestions lose to deadlines.

Limit	Value	Applies to	Rule
Method body length	**40 lines** hard, 20 target	every method	`S138`
Cyclomatic complexity	**10**	every method	`S1541`, `CA1502`
Parameters	**5**	every method, excluding `CancellationToken`	`S107`
Nesting depth	**3**	every method	`S134`
Methods per type	**15**	every type	`S1448`
File length	**300 lines**	every `.cs` file	`S104`
Class coupling	**30**	every type	`S1200`, `CA1506`

Additional shape rules that follow from the architecture:

- A CQRS handler has **exactly one** public method. If it needs a second, it is doing a second job.
- A controller action delegates to MediatR and returns. It contains no branching business logic.
- Constructors and DI registration methods are exempt from the **method-count** limit only. They are not exempt from length, complexity or nesting.

Enforcement and escape hatch

- `Microsoft.CodeAnalysis.NetAnalyzers` covers complexity (`CA1502`), maintainability (`CA1505`) and class coupling (`CA1506`).

- **Line-count and method-count limits are not expressible in the Microsoft analyzers.** `SonarAnalyzer.CSharp` supplies `S138`, `S1541`, `S107`, `S134`, `S1448`, `S104` and `S1200`. It is a build-time analyzer package: no runtime code ships, and restore downloads it, which is consistent with subject IV.1 (no third-party **source** committed).
- The escape hatch is an explicit `#pragma warning disable <rule>` with a comment naming the reason, re-enabled immediately after. It is visible in review and countable in the codebase.
- Lowering a severity globally in `.editorconfig` to make a build pass is not an option. Change the limit through an ADR, or fix the code.

Formatting

- `.editorconfig` at the repository root is the single formatting authority.
- `dotnet format --verify-no-changes` runs in CI and blocks merge.
- Four-space indent, braces on their own line, file-scoped namespaces.

Checklist

- ☐ `TreatWarningsAsErrors` on, build produces no warnings
- ☐ Nullable annotations honest — no blanket `!` to silence the compiler
- ☐ All I/O paths async with a `CancellationToken`
- ☐ `dotnet format --verify-no-changes` passes
- ☐ No method over 40 lines, complexity over 10, or type over 15 methods
- ☐ Every `#pragma warning disable` carries a reason and is re-enabled
- ☐ `IDE1006` escalated to error; no naming deviation in the diff

References

- C# coding conventions — https://learn.microsoft.com/dotnet/csharp/fundamentals/coding-style/coding-conventions
- Framework design guidelines — https://learn.microsoft.com/dotnet/standard/design-guidelines/
- Central package management — https://learn.microsoft.com/nuget/consume-packages/central-package-management
- .NET code-quality analyzer rules — https://learn.microsoft.com/dotnet/fundamentals/code-analysis/quality-rules/
- SonarSource C# rules — https://rules.sonarsource.com/csharp/

Related

`../clean-architecture-cqrs/STANDARD.md` · `../../60-quality/testing/STANDARD.md`